

Отчёт по лабораторной работе 9

Программирование цикла. Обработка аргументов командной строки.

Вахиш Дуаа Иссам Али Ахмед

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
2.1	Реализация подпрограмм в NASM	6
2.2	Отладка программ с помощью GDB	10
2.3	Задание для самостоятельной работы	21
3	Выводы	27

Список иллюстраций

2.1	Программа в файле lab9-1.asm	7
2.2	Запуск программы lab9-1.asm	8
2.3	Программа в файле lab9-1.asm	9
2.4	Запуск программы lab9-1.asm	9
2.5	Программа в файле lab9-2.asm	10
2.6	Запуск программы lab9-2.asm в отладчике	11
2.7	Дизассемблированный код	12
2.8	Дизассемблированный код в режиме интел	13
2.9	Точка остановки	14
2.10	Изменение регистров	15
2.11	Изменение регистров	16
2.12	Изменение значения переменной	17
2.13	Вывод значения регистра	18
2.14	Вывод значения регистра	19
2.15	Программа в файле lab9-3.asm	20
2.16	Вывод значения регистра	21
2.17	Программа в файле task-1.asm	22
2.18	Запуск программы task-1.asm	23
2.19	Код с ошибкой в файле task-2.asm	24
2.20	Отладка task-2.asm	25
2.21	Код исправлен в файле task-2.asm	26
2.22	Проверка работы task-2.asm	26

Список таблиц

1 Цель работы

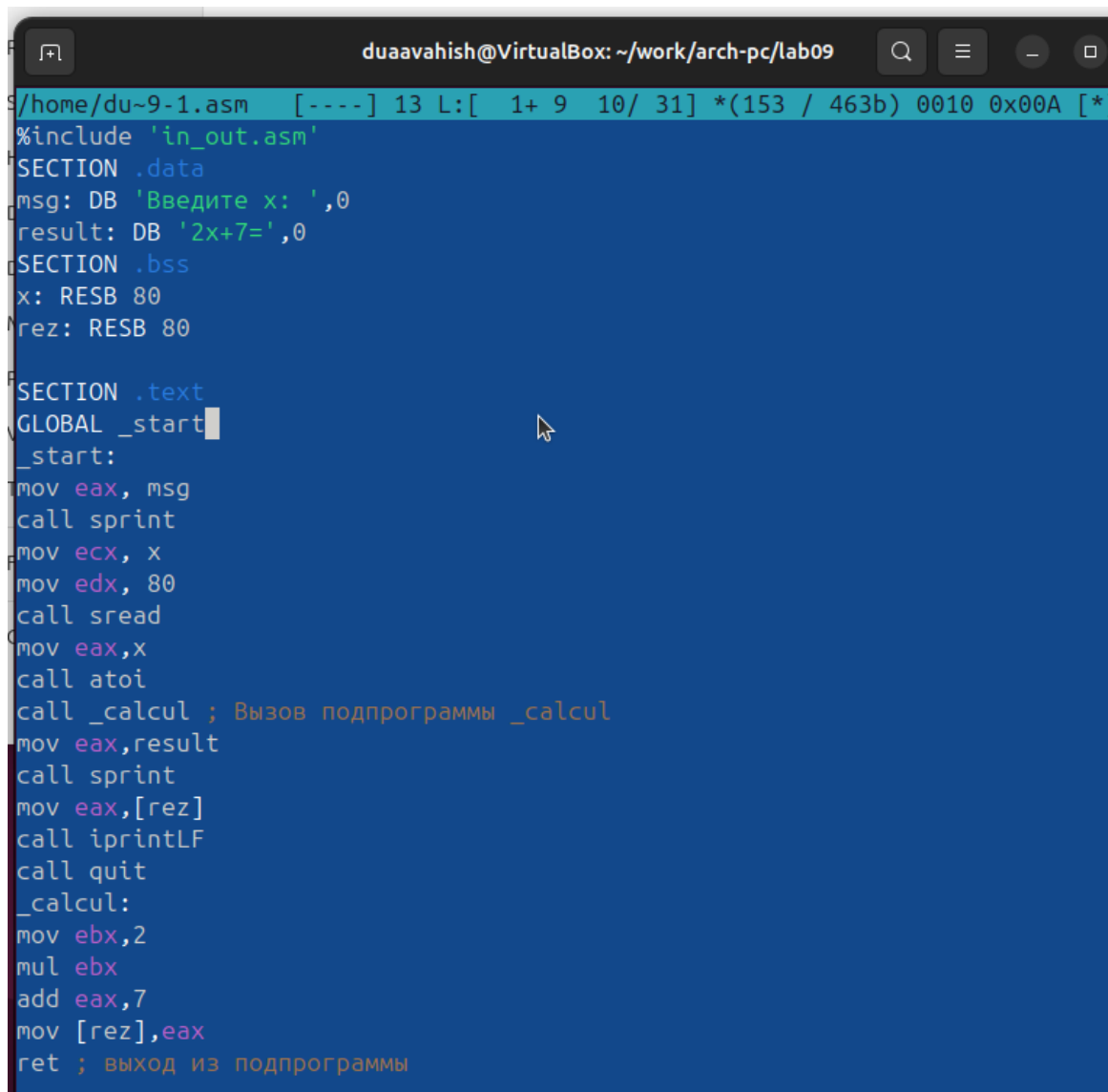
Целью работы является приобретение навыков написания программ с использованием подпрограмм. Знакомство с методами отладки при помощи GDB и его основными возможностями.

2 Выполнение лабораторной работы

2.1 Реализация подпрограмм в NASM

Я создала каталог для выполнения лабораторной работы №9 и перешла в него.

В качестве примера рассмотрим программу, которая вычисляет арифметическое выражение ($f(x) = 2x + 7$) с использованием подпрограммы `calcul`. В этом примере значение (x) вводится с клавиатуры, а само выражение вычисляется в подпрограмме.



```
duaaavahish@VirtualBox: ~/work/arch-pc/lab09
/home/du~9-1.asm [---] 13 L:[ 1+ 9 10/ 31] *(153 / 463b) 0010 0x00A [*]
%include 'in_out.asm'
SECTION .data
msg: DB 'Введите x: ',0
result: DB '2x+7=',0
SECTION .bss
x: RESB 80
rez: RESB 80

SECTION .text
GLOBAL _start
_start:
mov eax, msg
call sprint
mov ecx, x
mov edx, 80
call sread
mov eax,x
call atoi
call _calcul ; Вызов подпрограммы _calcul
mov eax,result
call sprint
mov eax,[rez]
call iprintLF
call quit
_calcul:
mov ebx,2
mul ebx
add eax,7
mov [rez],eax
ret ; выход из подпрограммы
```

Рисунок 2.1: Программа в файле lab9-1.asm

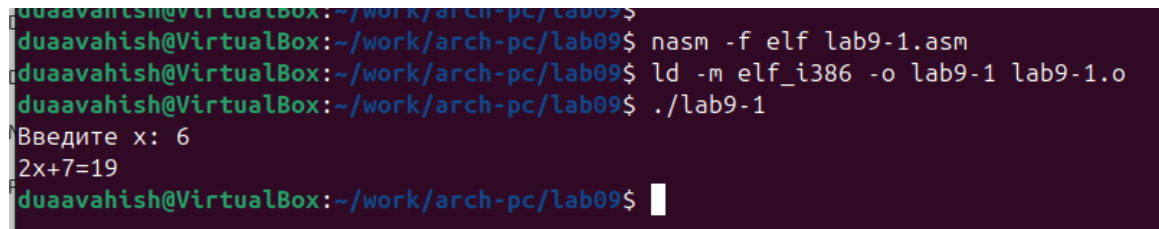
Первые строки программы отвечают за вывод сообщения на экран (с помощью вызова `sprint`), чтение данных, введенных с клавиатуры (с помощью вызова `sread`) и преобразование введенных данных из символьного вида в численный (с помощью вызова `atoi`).

После инструкции `call _calcul`, которая передает управление подпрограмме `_calcul`, будут выполнены инструкции, содержащиеся в подпрограмме.

Инструкция `ret` является последней в подпрограмме и её выполнение при-

водит к возврату в основную программу к инструкции, следующей за инструкцией `call`, которая вызвала данную подпрограмму.

Последние строки программы реализуют вывод сообщения (с помощью вызова `sprint`), вывод результата вычисления (с помощью вызова `iprintLF`) и завершение программы (с помощью вызова `quit`).



```
duaavahish@VirtualBox:~/work/arch-pc/lab09$  
duaavahish@VirtualBox:~/work/arch-pc/lab09$ nasm -f elf lab9-1.asm  
duaavahish@VirtualBox:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab9-1 lab9-1.o  
duaavahish@VirtualBox:~/work/arch-pc/lab09$ ./lab9-1  
Введите x: 6  
2x+7=19  
duaavahish@VirtualBox:~/work/arch-pc/lab09$
```

Рисунок 2.2: Запуск программы lab9-1.asm

Я изменила текст программы, добавив подпрограмму `subcalcul` в подпрограмму `calcul`, для вычисления выражения $(f(g(x)))$, где (x) вводится с клавиатуры, $(f(x) = 2x + 7, g(x) = 3x - 1)$.


```
duaaavahish@VirtualBox: ~/work/arch-pc/lab09
/home/du~9-1.asm [----] 9 L:[ 9+ 9 18/ 40] *(236 / 531b) 0010 0x00A

SECTION .text
GLOBAL _start
_start:
mov eax, msg
call sprint
mov ecx, x
mov edx, 80
call sread
mov eax, x
call atoi
call _calcul ; Вызов подпрограммы _calcul
mov eax, result
call sprint
mov eax, [rez]
call iprintLF
call quit

_calcul:
call _subcalcul
mov ebx, 2
mul ebx
add eax, 7
mov [rez], eax
ret ; выход из подпрограммы

_subcalcul:
mov ebx, 3
mul ebx
sub eax, 1
ret
```

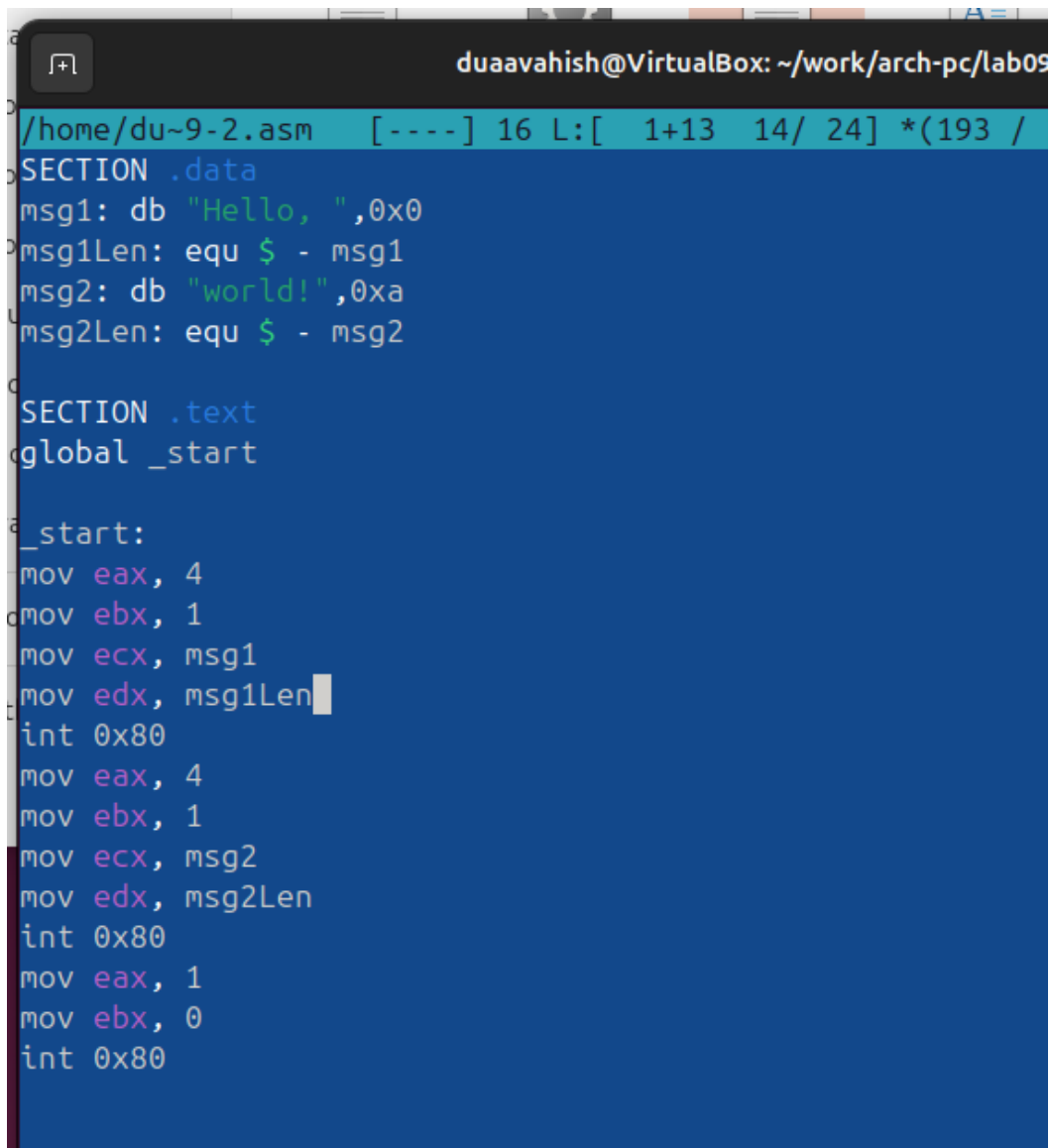
Рисунок 2.3: Программа в файле lab9-1.asm

```
duaaavahish@VirtualBox:~/work/arch-pc/lab09$ nasm -f elf lab9-1.asm
duaaavahish@VirtualBox:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab9-1 lab9-1.o
duaaavahish@VirtualBox:~/work/arch-pc/lab09$ ./lab9-1
Введите x: 6
2(3x-1)+7=41
duaaavahish@VirtualBox:~/work/arch-pc/lab09$
```

Рисунок 2.4: Запуск программы lab9-1.asm

2.2 Отладка программ с помощью GDB

Я создала файл lab9-2.asm с текстом программы из Листинга 9.2 (Программа печати сообщения Hello world!).



```
duaaavahish@VirtualBox: ~/work/arch-pc/lab09
/home/du~9-2.asm  [----] 16 L:[ 1+13 14/ 24] *(193 /
SECTION .data
msg1: db "Hello, ",0x0
msg1Len: equ $ - msg1
msg2: db "world!",0xa
msg2Len: equ $ - msg2

SECTION .text
global _start

_start:
mov eax, 4
mov ebx, 1
mov ecx, msg1
mov edx, msg1Len
int 0x80
mov eax, 4
mov ebx, 1
mov ecx, msg2
mov edx, msg2Len
int 0x80
mov eax, 1
mov ebx, 0
int 0x80
```

Рисунок 2.5: Программа в файле lab9-2.asm

После того как я получила исполняемый файл, для работы с GDB в исполняемый файл необходимо добавить отладочную информацию, для чего трансляцию программ следует проводить с ключом -g.

Загрузила исполняемый файл в отладчик GDB и проверила работу программы, запустив её в оболочке GDB с помощью команды run (сокращенно r).

```
duaavahish@VirtualBox:~/work/arch-pc/lab09$ nasm -f elf -g -l lab9-2.lst lab9-2.asm
duaavahish@VirtualBox:~/work/arch-pc/lab09$ ld -m elf_i386 -o lab9-2 lab9-2.o
duaavahish@VirtualBox:~/work/arch-pc/lab09$ gdb lab9-2
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab9-2...
(gdb) r
Starting program: /home/duaavahish/work/arch-pc/lab09/lab9-2

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n])
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
Hello, world!
[Inferior 1 (process 5005) exited normally]
(gdb)
```

Рисунок 2.6: Запуск программы lab9-2.asm в отладчике

Для более подробного анализа программы установила брейкпоинт на метку start, с которой начинается выполнение любой ассемблерной программы, и запустила её. Посмотрела дизассемблированный код программы.

```
duaavahish@VirtualBox: ~/work/arch-pc/lab09
(gdb) r
Starting program: /home/duaavahish/work/arch-pc/lab09/lab9-2

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n])
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
Hello, world!
[Inferior 1 (process 5005) exited normally]
(gdb) break _start
Breakpoint 1 at 0x08049000: file lab9-2.asm, line 11.
(gdb) r
Starting program: /home/duaavahish/work/arch-pc/lab09/lab9-2

Breakpoint 1, _start () at lab9-2.asm:11
11      mov eax, 4
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08049000 <+0>:      mov     $0x4,%eax
0x08049005 <+5>:      mov     $0x1,%ebx
0x0804900a <+10>:     mov     $0x804a000,%ecx
0x0804900f <+15>:     mov     $0x8,%edx
0x08049014 <+20>:     int     $0x80
0x08049016 <+22>:     mov     $0x4,%eax
0x0804901b <+27>:     mov     $0x1,%ebx
0x08049020 <+32>:     mov     $0x804a008,%ecx
0x08049025 <+37>:     mov     $0x7,%edx
0x0804902a <+42>:     int     $0x80
0x0804902c <+44>:     mov     $0x1,%eax
0x08049031 <+49>:     mov     $0x0,%ebx
0x08049036 <+54>:     int     $0x80
End of assembler dump.
(gdb) 
```

Рисунок 2.7: Дизассемблированный код

```
duaavahish@VirtualBox: ~/work/arch-pc/lab09
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08049000 <+0>:      mov     $0x4,%eax
    0x08049005 <+5>:      mov     $0x1,%ebx
    0x0804900a <+10>:     mov     $0x804a000,%ecx
    0x0804900f <+15>:     mov     $0x8,%edx
    0x08049014 <+20>:     int     $0x80
    0x08049016 <+22>:     mov     $0x4,%eax
    0x0804901b <+27>:     mov     $0x1,%ebx
    0x08049020 <+32>:     mov     $0x804a008,%ecx
    0x08049025 <+37>:     mov     $0x7,%edx
    0x0804902a <+42>:     int     $0x80
    0x0804902c <+44>:     mov     $0x1,%eax
    0x08049031 <+49>:     mov     $0x0,%ebx
    0x08049036 <+54>:     int     $0x80
End of assembler dump.
(gdb) set disassembly-flavor intel
(gdb) disassemble _start
Dump of assembler code for function _start:
=> 0x08049000 <+0>:      mov     eax,0x4
    0x08049005 <+5>:      mov     ebx,0x1
    0x0804900a <+10>:     mov     ecx,0x804a000
    0x0804900f <+15>:     mov     edx,0x8
    0x08049014 <+20>:     int     0x80
    0x08049016 <+22>:     mov     eax,0x4
    0x0804901b <+27>:     mov     ebx,0x1
    0x08049020 <+32>:     mov     ecx,0x804a008
    0x08049025 <+37>:     mov     edx,0x7
    0x0804902a <+42>:     int     0x80
    0x0804902c <+44>:     mov     eax,0x1
    0x08049031 <+49>:     mov     ebx,0x0
    0x08049036 <+54>:     int     0x80
End of assembler dump.
(gdb) 
```

Рисунок 2.8: Дизассемблированный код в режиме интел

Для установки точки останова использовала команду `break` (кратко `b`). Типичный аргумент этой команды — место установки. Его можно задать либо как номер строки программы (если есть исходный файл и программа компилировалась с отладочной информацией), либо как имя метки, или как адрес. Чтобы избежать путаницы с номерами, перед адресом ставится «звездочка».

На предыдущих шагах была установлена точка останова по имени метки `_start`. Проверила это с помощью команды `info breakpoints` (кратко `i b`). Затем

установила ещё одну точку останова по адресу инструкции. Адрес инструкции можно увидеть в средней части экрана в левом столбце соответствующей инструкции. Определила адрес предпоследней инструкции (mov ebx,0x0) и установила точку.

```

duaavahish@VirtualBox: ~/work/arch-pc/lab09
Register group: general
eax      0x0      0
ecx      0x0      0
edx      0x0      0
ebx      0x0      0
esp      0xffffcee0 0xffffcee0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8049000 0x8049000 <_start>

B> 0x8049000 <_start>  mov  eax,0x4
   0x8049005 <_start+5> mov  ebx,0x1
   0x804900a <_start+10> mov  ecx,0x804a000
   0x804900f <_start+15> mov  edx,0x8
   0x8049014 <_start+20> int  0x80
   0x8049016 <_start+22> mov  eax,0x4
   0x804901b <_start+27> mov  ebx,0x1
   0x8049020 <_start+32> mov  ecx,0x804a008
   0x8049025 <_start+37> mov  edx,0x7
   0x804902a <_start+42> int  0x80

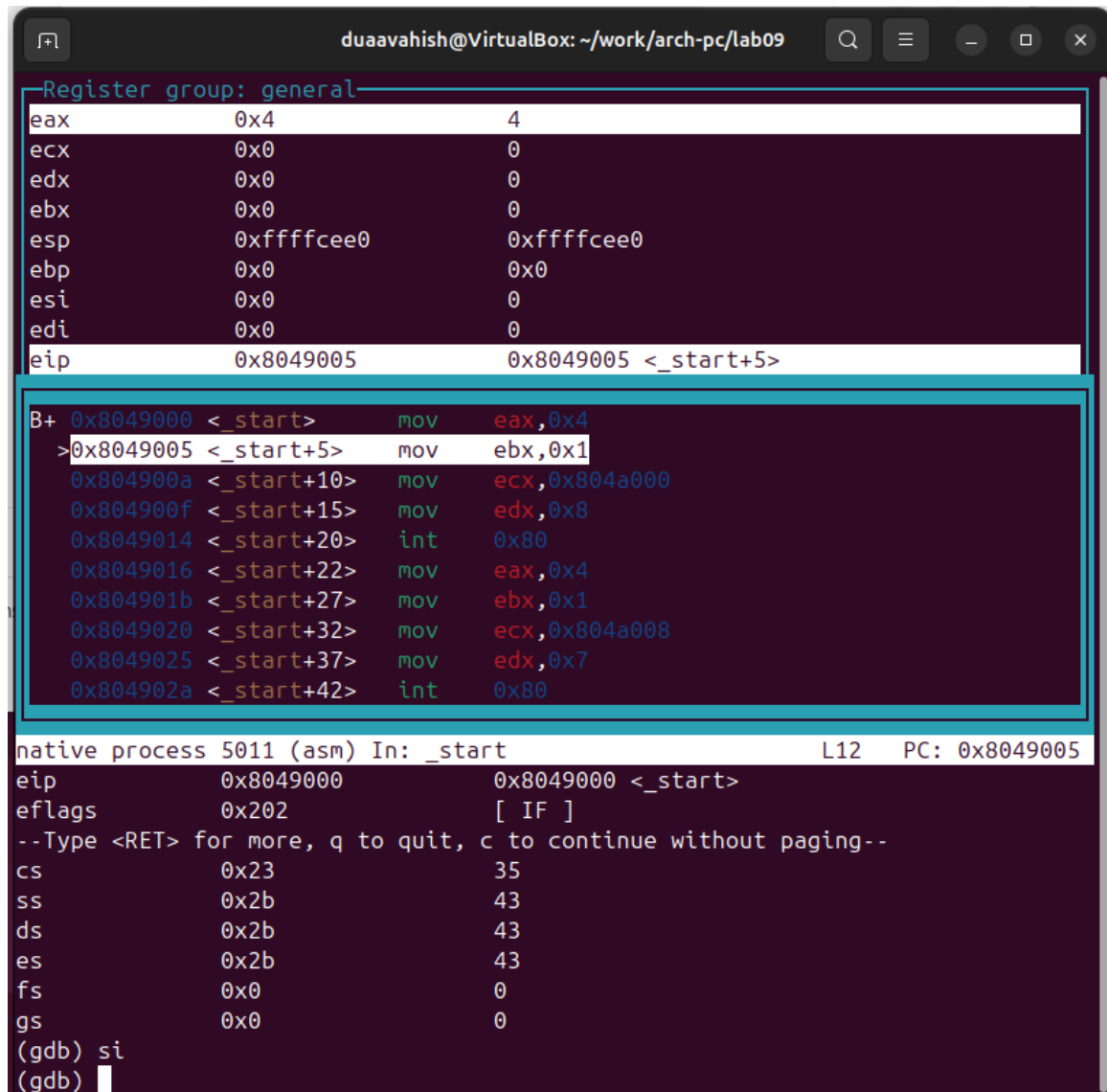
native process 5011 (asm) In: _start L11 PC: 0x8049000
(gdb) layout regs
(gdb) b *0x8049031
Breakpoint 2 at 0x8049031: file lab9-2.asm, line 22.
(gdb) i b
Num    Type           Disp Enb Address      What
1      breakpoint     keep y   0x08049000 lab9-2.asm:11
       breakpoint already hit 1 time
2      breakpoint     keep y   0x08049031 lab9-2.asm:22
(gdb)

```

Рисунок 2.9: Точка останова

Отладчик может показывать содержимое ячеек памяти и регистров, а при необходимости позволяет вручную изменять значения регистров и переменных. Я выполнила 5 инструкций с помощью команды stepi (или si) и просле-

дила за изменением значений регистров.



The screenshot shows a GDB debugger window titled "duaavahish@VirtualBox: ~/work/arch-pc/lab09". The window is divided into several sections. The top section, titled "Register group: general", displays the current values of the general-purpose registers: eax (0x4), ecx (0x0), edx (0x0), ebx (0x0), esp (0xffffcee0), ebp (0x0), esi (0x0), edi (0x0), and eip (0x8049005). Below this, a list of assembly instructions is shown, with the instruction at address 0x8049005, "mov ebx, 0x1", highlighted. The bottom section shows the state of the native process 5011 (asm) at the start of the program, including the current instruction pointer (eip) at 0x8049000, the instruction "mov eax, 0x4", and the current instruction pointer (eip) at 0x8049005. The window also displays the current instruction pointer (eip) at 0x8049005 and the current instruction "mov ebx, 0x1".

```
Register group: general
eax      0x4      4
ecx      0x0      0
edx      0x0      0
ebx      0x0      0
esp      0xffffcee0 0xffffcee0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8049005 0x8049005 <_start+5>

B+ 0x8049000 <_start>    mov    eax,0x4
>0x8049005 <_start+5>    mov    ebx,0x1
0x804900a <_start+10>    mov    ecx,0x804a000
0x804900f <_start+15>    mov    edx,0x8
0x8049014 <_start+20>    int     0x80
0x8049016 <_start+22>    mov    eax,0x4
0x804901b <_start+27>    mov    ebx,0x1
0x8049020 <_start+32>    mov    ecx,0x804a008
0x8049025 <_start+37>    mov    edx,0x7
0x804902a <_start+42>    int     0x80

native process 5011 (asm) In: _start L12 PC: 0x8049005
eip      0x8049000 0x8049000 <_start>
eflags   0x202    [ IF ]
--Type <RET> for more, q to quit, c to continue without paging--
cs       0x23     35
ss       0x2b     43
ds       0x2b     43
es       0x2b     43
fs       0x0      0
gs       0x0      0
(gdb) si
(gdb)
```

Рисунок 2.10: Изменение регистров

```
duaavahish@VirtualBox: ~/work/arch-pc/lab09
Register group: general
eax      0x8      8
ecx      0x804a000 134520832
edx      0x8      8
ebx      0x1      1
esp      0xffffcee0 0xffffcee0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8049016 0x8049016 <_start+22>

B+ 0x8049000 <_start>      mov    eax,0x4
   0x8049005 <_start+5>    mov    ebx,0x1
   0x804900a <_start+10>   mov    ecx,0x804a000
   0x804900f <_start+15>   mov    edx,0x8
   0x8049014 <_start+20>   int    0x80
> 0x8049016 <_start+22>   mov    eax,0x4
   0x804901b <_start+27>   mov    ebx,0x1
   0x8049020 <_start+32>   mov    ecx,0x804a008
   0x8049025 <_start+37>   mov    edx,0x7
   0x804902a <_start+42>   int    0x80

native process 5011 (asm) In: _start      L16      PC: 0x8049016
es      0x2b      43
fs      0x0      0
gs      0x0      0
(gdb) si
(gdb) si
(gdb) si
(gdb) so
source command requires file name of file to source.
(gdb) si
(gdb) si
(gdb) █
```

Рисунок 2.11: Изменение регистров

Посмотрела значение переменной msg1 по имени и значение переменной msg2 по адресу.

Изменить значение для регистра или ячейки памяти можно с помощью команды set, указав имя регистра или адрес. Я изменила первый символ переменной msg1.


```
duaavahish@VirtualBox: ~/work/arch-pc/lab09
Register group: general
eax      0x8      8
ecx      0x804a000 134520832
edx      0x8      8
ebx      0x1      1
esp      0xffffcee0 0xffffcee0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8049016 0x8049016 <_start+22>

B+ 0x8049000 <_start>    mov    eax,0x4
   0x8049005 <_start+5>  mov    ebx,0x1
   0x804900a <_start+10> mov    ecx,0x804a000
   0x804900f <_start+15> mov    edx,0x8
   0x8049014 <_start+20> int     0x80
>0x8049016 <_start+22>  mov    eax,0x4
   0x804901b <_start+27>  mov    ebx,0x1
   0x8049020 <_start+32> mov    ecx,0x804a008
   0x8049025 <_start+37> mov    edx,0x7
   0x804902a <_start+42> int     0x80

native process 5011 (asm) In: _start      L16    PC: 0x8049016
0x804a000 <msg1>:      "Hello, "
0x804a008 <msg2>:      "world!\n\034"
(gdb) x/1sb 0x804a008
0x804a008 <msg2>:      "world!\n\034"
(gdb) set {char}&msg1='h'
(gdb) x/1sb &msg1
0x804a000 <msg1>:      "hello, "
(gdb) set {char}0x804a008='L'
(gdb) x/1sb 0x804a008
0x804a008 <msg2>:      "Lorld!\n\034"
(gdb) 
```

Рисунок 2.12: Изменение значения переменной

Я вывела значение регистра `edx` в различных форматах (в шестнадцатеричном, двоичном и символьном).

```
duaavahish@VirtualBox: ~/work/arch-pc/lab09
Register group: general
eax      0x8      8
ecx      0x804a000 134520832
edx      0x8      8
ebx      0x1      1
esp      0xffffcee0 0xffffcee0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8049016 0x8049016 <_start+22>

B+ 0x8049000 <_start>    mov    eax,0x4
0x8049005 <_start+5>    mov    ebx,0x1
0x804900a <_start+10>   mov    ecx,0x804a000
0x804900f <_start+15>   mov    edx,0x8
0x8049014 <_start+20>   int    0x80
>0x8049016 <_start+22>  mov    eax,0x4
0x804901b <_start+27>   mov    ebx,0x1
0x8049020 <_start+32>   mov    ecx,0x804a008
0x8049025 <_start+37>   mov    edx,0x7
0x804902a <_start+42>   int    0x80

native process 5011 (asm) In: _start L16 PC: 0x8049016
(gdb) p/s $ecx
$3 = 134520832
(gdb) p/x $ecx
$4 = 0x804a000
(gdb) p/s $edx
$5 = 8
(gdb) p/t $edx
$6 = 1000
(gdb) p/x $edx
$7 = 0x8
(gdb) 
```

Рисунок 2.13: Вывод значения регистра

С помощью команды set изменила значение регистра ebx.

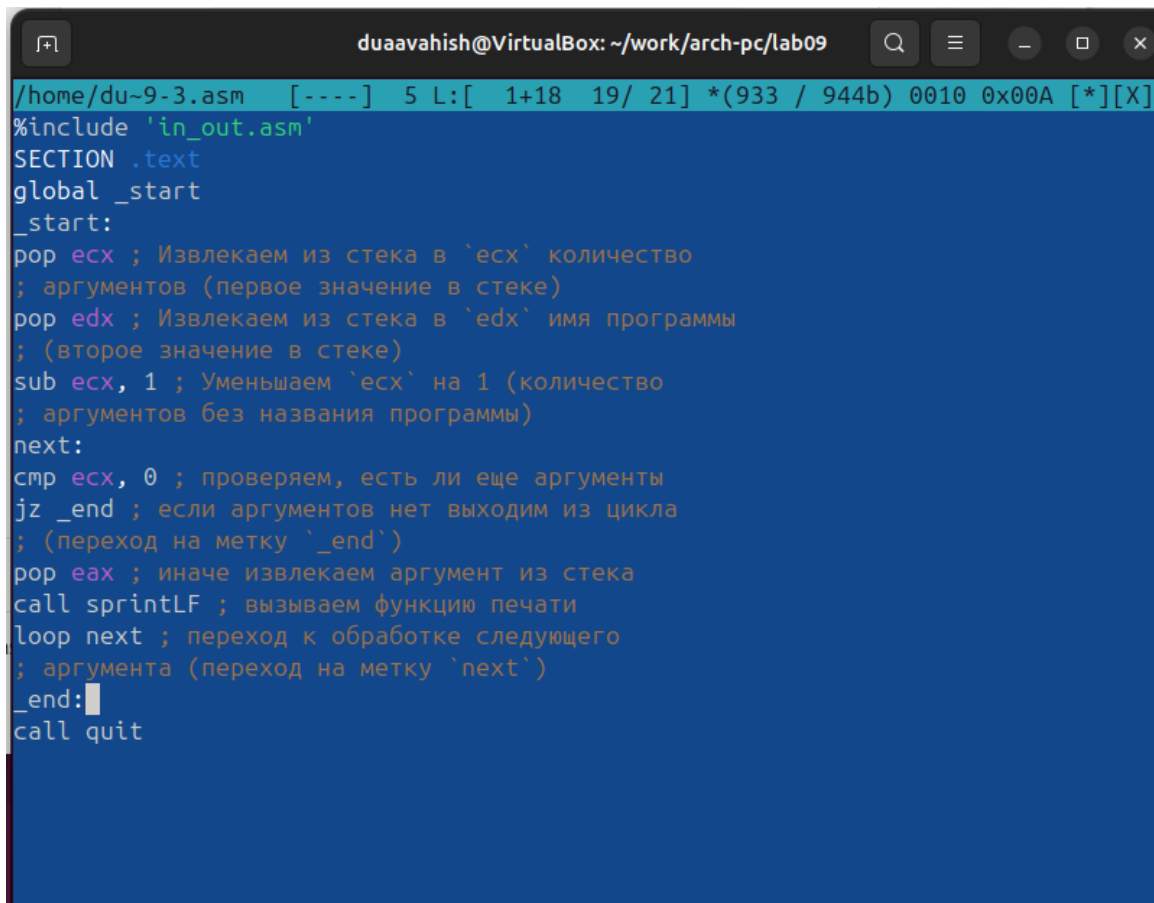
```
duaavahish@VirtualBox: ~/work/arch-pc/lab09
Register group: general
eax      0x8      8
ecx      0x804a000 134520832
edx      0x8      8
ebx      0x2      2
esp      0xffffcee0 0xffffcee0
ebp      0x0      0x0
esi      0x0      0
edi      0x0      0
eip      0x8049016 0x8049016 <_start+22>

B+ 0x8049000 <_start>    mov    eax,0x4
    0x8049005 <_start+5>  mov    ebx,0x1
    0x804900a <_start+10> mov    ecx,0x804a000
    0x804900f <_start+15> mov    edx,0x8
    0x8049014 <_start+20> int     0x80
>0x8049016 <_start+22> mov    eax,0x4
    0x804901b <_start+27> mov    ebx,0x1
    0x8049020 <_start+32> mov    ecx,0x804a008
    0x8049025 <_start+37> mov    edx,0x7
    0x804902a <_start+42> int     0x80

native process 5011 (asm) In: _start      L16    PC: 0x8049016
(gdb) p/t $edx
$6 = 1000
(gdb) p/x $edx
$7 = 0x8
(gdb) set $ebx='2'
(gdb) p/s $ebx
$8 = 50
(gdb) set $ebx=2
(gdb) p/s $ebx
$9 = 2
(gdb) 
```

Рисунок 2.14: Вывод значения регистра

Я скопировала файл lab8-2.asm, созданный при выполнении лабораторной работы №8, с программой, выводящей на экран аргументы командной строки. Создала исполняемый файл. Для загрузки программы с аргументами в GDB необходимо использовать ключ `-args`. Загрузила исполняемый файл в отладчик, указав аргументы.



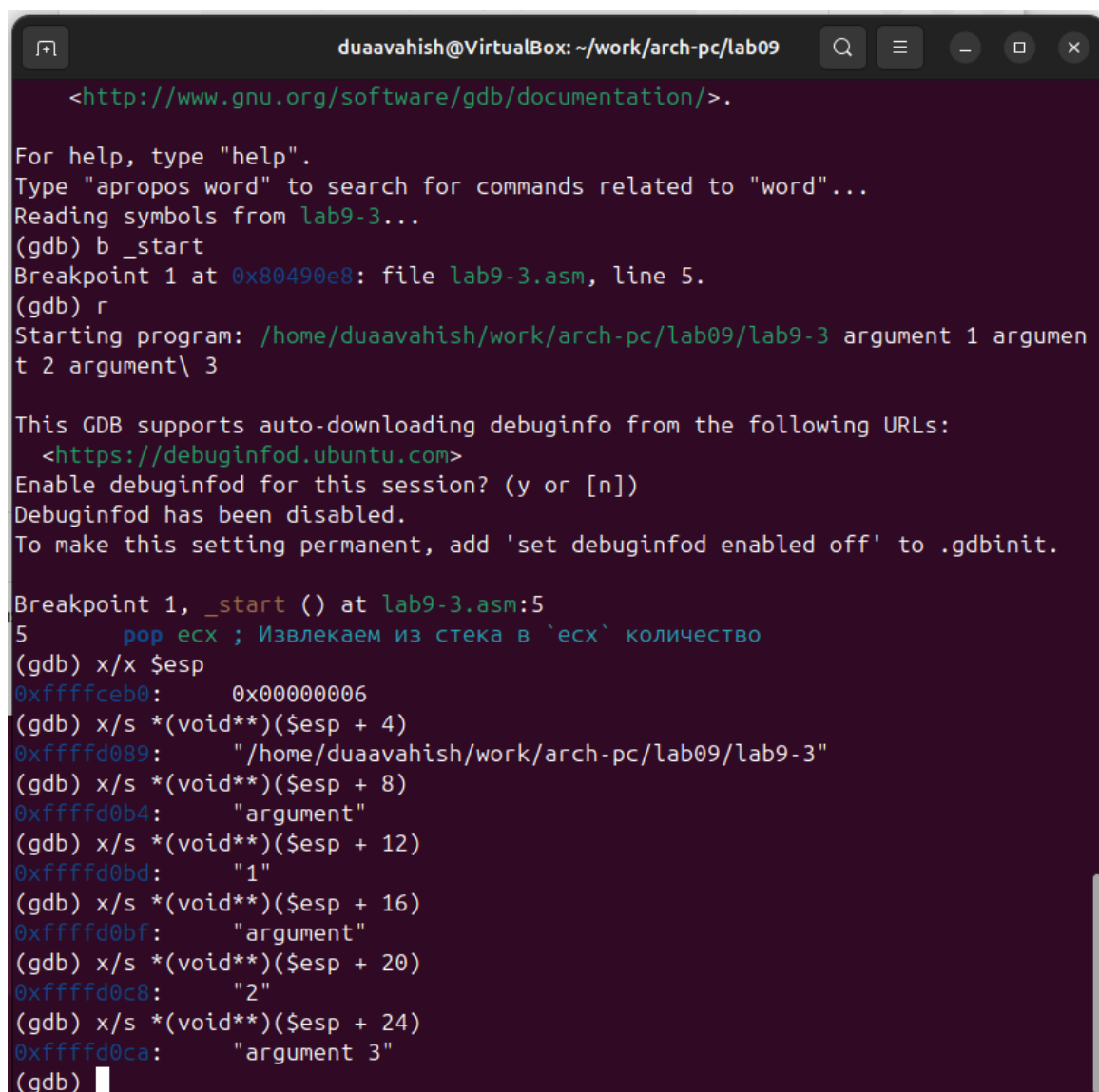
```
/home/du~9-3.asm [----] 5 L: [ 1+18 19/ 21] *(933 / 944b) 0010 0x00A [*][X]
#include 'in_out.asm'
SECTION .text
global _start
_start:
pop ecx ; Извлекаем из стека в `ecx` количество
; аргументов (первое значение в стеке)
pop edx ; Извлекаем из стека в `edx` имя программы
; (второе значение в стеке)
sub ecx, 1 ; Уменьшаем `ecx` на 1 (количество
; аргументов без названия программы)
next:
cmp ecx, 0 ; проверяем, есть ли еще аргументы
jz _end ; если аргументов нет выходим из цикла
; (переход на метку `_end`)
pop eax ; иначе извлекаем аргумент из стека
call sprintf ; вызываем функцию печати
loop next ; переход к обработке следующего
; аргумента (переход на метку `next`)
_end:
call quit
```

Рисунок 2.15: Программа в файле lab9-3.asm

Для начала установила точку останова перед первой инструкцией в программе и запустила её.

Адрес вершины стека хранится в регистре `esp`, и по этому адресу располагается число, равное количеству аргументов командной строки (включая имя программы). Как видно, число аргументов равно 5 — это имя программы `lab9-3` и непосредственно аргументы: `аргумент1`, `аргумент2` и `аргумент 3`.

Посмотрела остальные позиции стека — по адресу `[esp+4]` располагается адрес в памяти, где находится имя программы, по адресу `[esp+8]` — адрес первого аргумента, по адресу `[esp+12]` — второго и т.д.



```
duaavahish@VirtualBox: ~/work/arch-pc/lab09
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from lab9-3...
(gdb) b _start
Breakpoint 1 at 0x80490e8: file lab9-3.asm, line 5.
(gdb) r
Starting program: /home/duaavahish/work/arch-pc/lab09/lab9-3 argument 1 argumen
t 2 argument\ 3

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n])
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.

Breakpoint 1, _start () at lab9-3.asm:5
5      pop ecx ; Извлекаем из стека в `ecx` количество
(gdb) x/x $esp
0xffffceb0: 0x00000006
(gdb) x/s *(void**)($esp + 4)
0xffffd089: "/home/duaavahish/work/arch-pc/lab09/lab9-3"
(gdb) x/s *(void**)($esp + 8)
0xffffd0b4: "argument"
(gdb) x/s *(void**)($esp + 12)
0xffffd0bd: "1"
(gdb) x/s *(void**)($esp + 16)
0xffffd0bf: "argument"
(gdb) x/s *(void**)($esp + 20)
0xffffd0c8: "2"
(gdb) x/s *(void**)($esp + 24)
0xffffd0ca: "argument 3"
(gdb)
```

Рисунок 2.16: Вывод значения регистра

Объяснила, почему шаг изменения адреса равен 4 ([esp+4], [esp+8], [esp+12]) — шаг равен размеру переменной (4 байта).

2.3 Задание для самостоятельной работы

Я переписала программу из лабораторной работы №8, чтобы вычислить значение функции ($f(x)$) в виде подпрограммы.

```
duaaavahish@VirtualBox: ~/v
/home/du~k-1.asm [----] 0 L:[ 7+16 23/
global _start
_start:
mov eax, fx
call sprintLF
pop ecx
pop edx
sub ecx,1
mov esi, 0

next:
cmp ecx,0h
jz _end
pop eax
call atoi
call prog
add esi,eax
loop next

_end:
mov eax, msg
call sprint
mov eax, esi
call iprintLF
call quit

prog:
mov ebx,4
mul ebx
sub eax,3
ret
```

Рисунок 2.17: Программа в файле task-1.asm

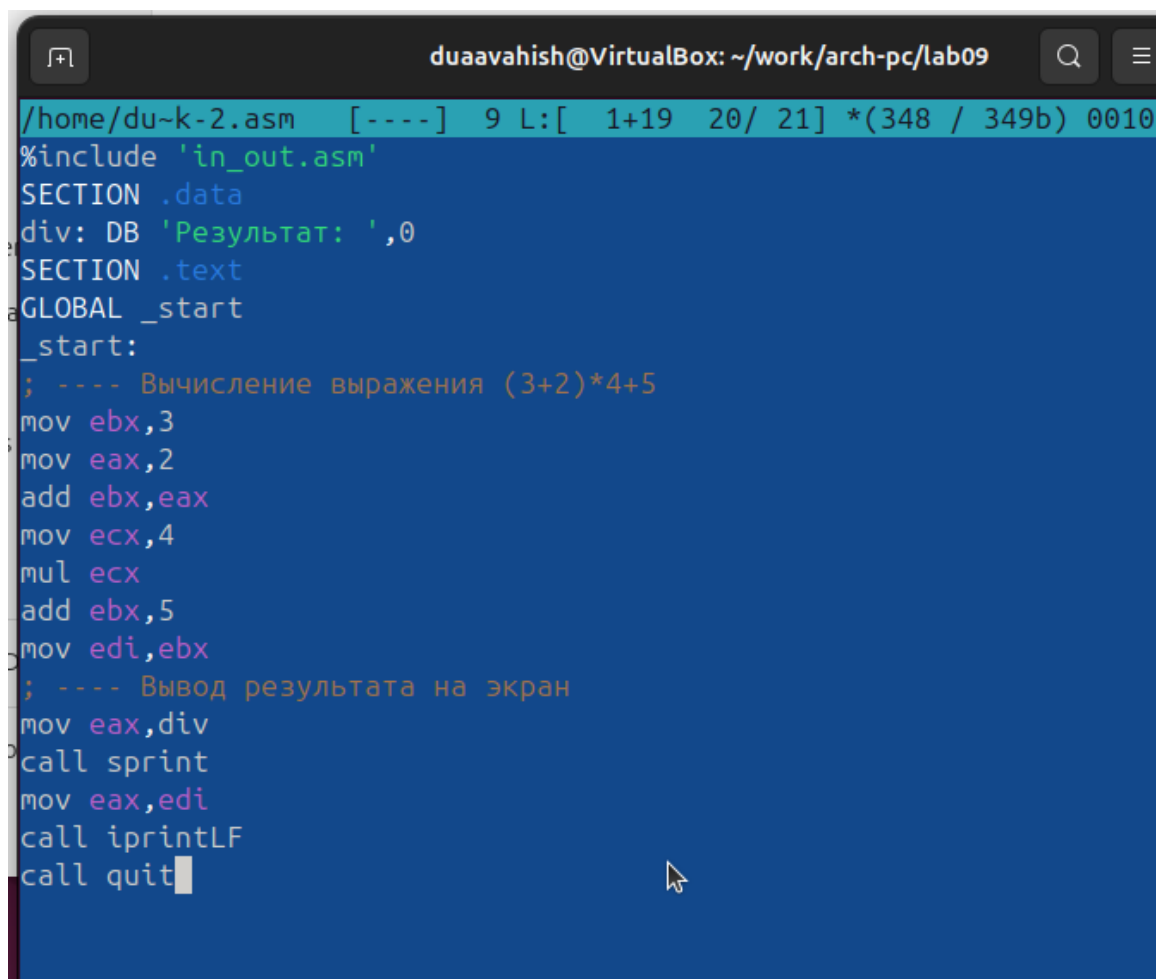
```

duaavahish@VirtualBox:~/work/arch-pc/lab09$
duaavahish@VirtualBox:~/work/arch-pc/lab09$ nasm -f elf task-1.asm
duaavahish@VirtualBox:~/work/arch-pc/lab09$ ld -m elf_i386 task-1.o -o task-1
duaavahish@VirtualBox:~/work/arch-pc/lab09$ ./task-1
f(x)= 4x - 3
Результат: 0
duaavahish@VirtualBox:~/work/arch-pc/lab09$ ./task-1 2
f(x)= 4x - 3
Результат: 5
duaavahish@VirtualBox:~/work/arch-pc/lab09$ ./task-1 2 3 4 5 6
f(x)= 4x - 3
Результат: 65
duaavahish@VirtualBox:~/work/arch-pc/lab09$

```

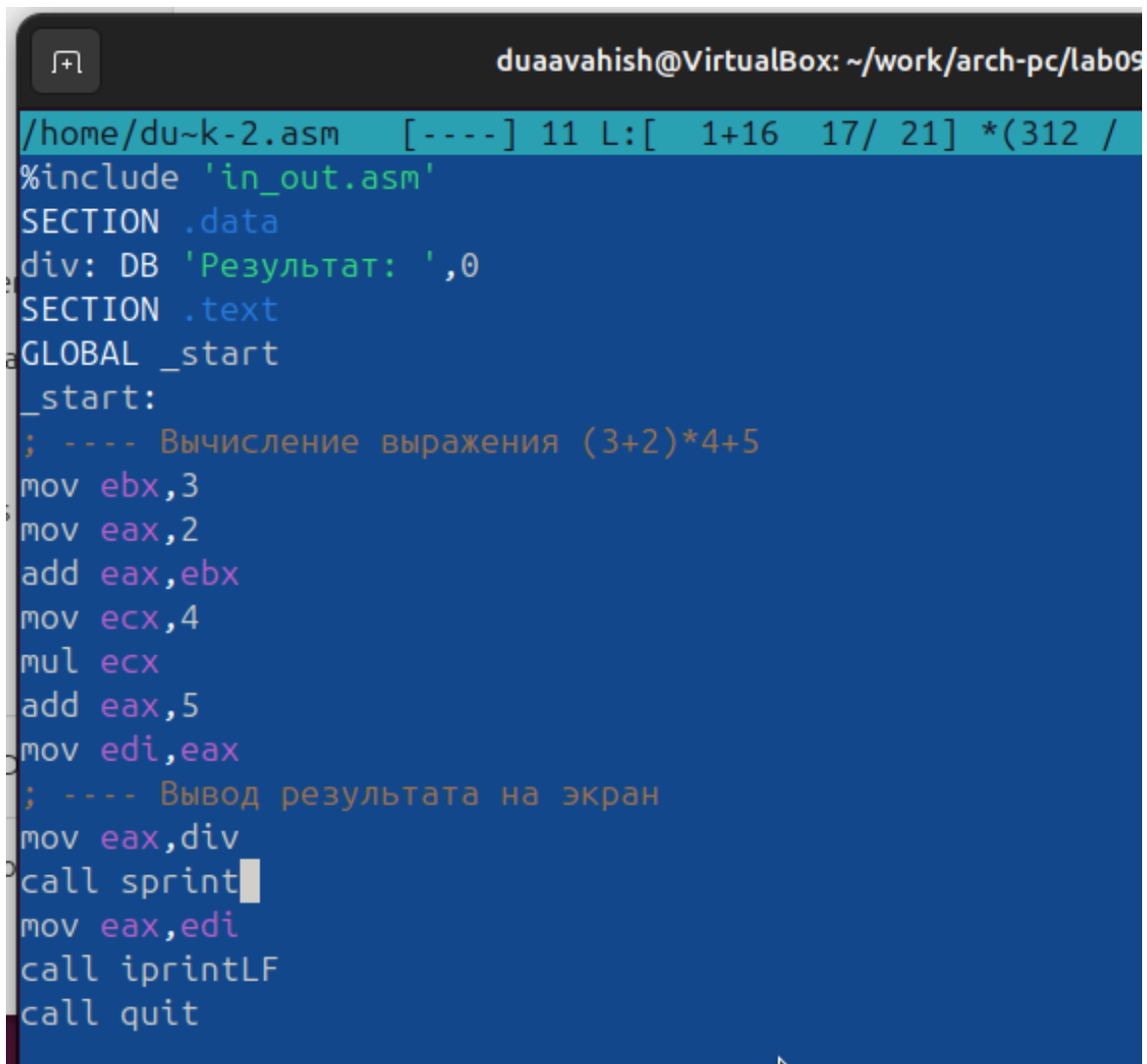
Рисунок 2.18: Запуск программы task-1.asm

Приведенный ниже листинг программы вычисляет выражение $((3+2)*4+5)$. Однако при запуске программа дает неверный результат. Я проверила это и решила использовать отладчик GDB для анализа изменений значений регистров и определения ошибки.



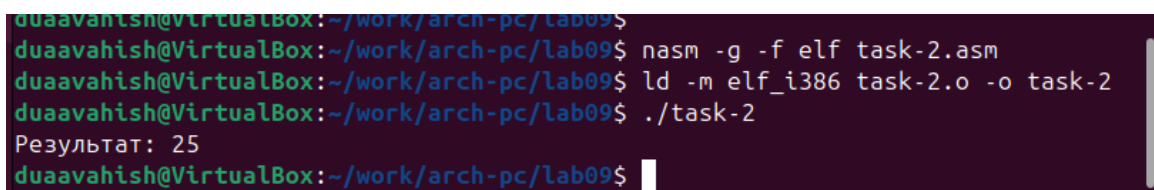
```
duaavahish@VirtualBox: ~/work/arch-pc/lab09
/home/duaavahish/task-2.asm [----] 9 L:[ 1+19 20/ 21] *(348 / 349b) 0010
#include 'in_out.asm'
SECTION .data
div: DB 'Результат: ',0
SECTION .text
GLOBAL _start
_start:
; ---- Вычисление выражения (3+2)*4+5
mov ebx,3
mov eax,2
add ebx,eax
mov ecx,4
mul ecx
add ebx,5
mov edi,ebx
; ---- Вывод результата на экран
mov eax,div
call sprint
mov eax,edi
call iprintLF
call quit
```

Рисунок 2.19: Код с ошибкой в файле task-2.asm



```
duaavahish@VirtualBox: ~/work/arch-pc/lab09
/home/duaavahish/task-2.asm [----] 11 L:[ 1+16 17/ 21] *(312 /
#include 'in_out.asm'
SECTION .data
div: DB 'Результат: ',0
SECTION .text
GLOBAL _start
_start:
; ---- Вычисление выражения (3+2)*4+5
mov ebx,3
mov eax,2
add eax,ebx
mov ecx,4
mul ecx
add eax,5
mov edi,eax
; ---- Вывод результата на экран
mov eax,div
call sprint
mov eax,edi
call iprintLF
call quit
```

Рисунок 2.21: Код исправлен в файле task-2.asm



```
duaavahish@VirtualBox:~/work/arch-pc/lab09$
duaavahish@VirtualBox:~/work/arch-pc/lab09$ nasm -g -f elf task-2.asm
duaavahish@VirtualBox:~/work/arch-pc/lab09$ ld -m elf_i386 task-2.o -o task-2
duaavahish@VirtualBox:~/work/arch-pc/lab09$ ./task-2
Результат: 25
duaavahish@VirtualBox:~/work/arch-pc/lab09$
```

Рисунок 2.22: Проверка работы task-2.asm

3 Выводы

Освоили работу с подпрограммами и отладчиком.